

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 01 -

BOAS PRÁTICAS DE CÓDIGO LIMPO EM ML

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	14
O QUE VOCÊ VIU NESTA AULA?	17
REFERÊNCIAS.....	19

EMANIP

O QUE VEM POR AÍ?

Ao longo desta aula, vamos discutir como o código em projetos de Machine Learning vai além de simplesmente “funcionar”. Vimos que escrever código legível, simples e bem estruturado é essencial para garantir confiabilidade, escalabilidade e compreensão dos modelos desenvolvidos. Conceitos como Clean Code, nomes significativos, simplicidade (KISS), evitar duplicações (DRY) e a prática de code review mostram que o código é um meio de comunicação entre pessoas, não apenas um conjunto de instruções para a máquina.

Nessa aula, esses fundamentos servirão como base para avançar em temas mais próximos da Engenharia de Machine Learning, em que o código passa a sustentar pipelines completos, modelos em produção e processos contínuos de evolução. O convite agora é olhar para o que foi visto não como regras isoladas, mas como princípios que orientam decisões técnicas e tornam soluções de Machine Learning mais seguras, profissionais e preparadas para o mundo real.

HANDS ON

Em projetos de Machine Learning, grande parte da atenção costuma estar voltada para algoritmos, métricas e desempenho dos modelos. No entanto, existe um elemento central que sustenta todas essas decisões e que, muitas vezes, passa despercebido: o código. É o código que define como os dados são preparados, como o modelo aprende, como os resultados são avaliados e se todo esse processo pode ser compreendido e reproduzido ao longo do tempo. Esta aula convida você a olhar para o código como parte essencial da qualidade de qualquer solução em Machine Learning.

Ao longo desta leitura, você perceberá que fazer o código funcionar não é o mesmo que escrever um bom código. Em Machine Learning, códigos confusos podem gerar modelos aparentemente corretos, mas conceitualmente frágeis. Pequenos erros de implementação, duplicações ou decisões mal estruturadas podem comprometer previsões e gerar resultados que não se sustentam quando o modelo é colocado em uso real. Por isso, aprender a escrever código claro, simples e organizado é uma habilidade tão importante quanto dominar técnicas de modelagem.

Nesta aula, você terá contato com os princípios de Clean Code aplicados diretamente ao contexto de Machine Learning. Serão discutidos temas como legibilidade do código, nomes significativos, simplicidade (KISS), evitar duplicações (DRY) e a importância de estruturar pipelines de forma clara e coerente. A ideia central é mostrar que o código é um meio de comunicação entre pessoas, capaz de tornar explícitas as decisões técnicas e estatísticas envolvidas na construção de modelos. Também será abordada a prática de code review, destacando seu papel na identificação de erros silenciosos, na padronização do código e no fortalecimento da qualidade das soluções.

O objetivo desta aula é provocar uma mudança de perspectiva: enxergar o código não apenas como um meio de execução, mas como um elemento fundamental para construir soluções de Machine Learning confiáveis, escaláveis e preparadas para o mundo real. Ao avançar nesta leitura, você desenvolverá uma base sólida para evoluir de scripts experimentais para pipelines bem estruturados e sustentáveis.

SAIBA MAIS

Em Machine Learning, o código é o elemento responsável por extrair padrões dos dados e gerar previsões. Por meio dele, carregamos bases de dados, realizamos transformações, criamos variáveis, treinamos modelos, ajustamos parâmetros e avaliamos resultados. Na prática, isso envolve dezenas ou centenas de linhas de código, muitas vezes escritas rapidamente para testar hipóteses ou validar ideias.

O problema surge quando esse código cresce sem organização. Variáveis com nomes genéricos, funções extensas que acumulam múltiplas responsabilidades, duplicação de lógica e estruturas difíceis de interpretar tornam o projeto difícil de entender e manter. Em Machine Learning, esse risco é ainda maior, pois o código não apenas executa operações, mas define diretamente como o modelo aprende. Pequenos erros de implementação podem gerar previsões aparentemente corretas, mas estatisticamente inválidas.

Nesse contexto, surge o Clean Code, que consiste na prática de escrever código claro, simples e bem estruturado, tornando explícitas as transformações realizadas, a lógica do modelo e as decisões tomadas ao longo do pipeline. Um código limpo permite compreender rapidamente o que foi feito, por que foi feito e como o processo pode ser modificado ou evoluído.

Projetos de Machine Learning são particularmente sensíveis à qualidade do código, pois seus resultados dependem diretamente da sequência de etapas implementadas. Quando carregamento de dados, pré-processamento, treinamento e avaliação são misturados em um único fluxo, a depuração torna-se difícil, a reprodutibilidade é comprometida e o risco de erros conceituais, como vazamento de dados ou avaliações incorretas, aumenta significativamente.

Portanto, aplicar Clean Code em Machine Learning não é apenas uma questão estética ou organizacional, mas uma prática essencial para proteger a qualidade das previsões, garantir confiança nos resultados e tornar os modelos sustentáveis ao longo do tempo. O objetivo é transformar scripts experimentais em pipelines organizados, legíveis e preparados para uso profissional.

Quando discutimos Clean Code, não estamos tratando apenas de estilo, mas da diferença entre um código bem construído e um código apenas funcional. Código

funcional é aquele que executa sua tarefa e produz um resultado. No entanto, esse tipo de código frequentemente esconde decisões importantes, mistura responsabilidades e dificulta a compreensão do processo de aprendizado do modelo. O Clean Code torna esse processo explícito, estruturado e rastreável.

Essa distinção é especialmente relevante em Machine Learning, pois um código que funciona não é necessariamente confiável. Modelos podem apresentar boas métricas mesmo quando foram treinados incorretamente, devido a transformações inadequadas, vazamento de dados ou avaliações mal conduzidas. Ao aplicar princípios como separação de responsabilidades, nomes significativos e organização clara do pipeline, o foco deixa de ser apenas a execução e passa a ser a construção de um processo reproduzível, auditável e confiável.

Código que funciona × código que escala, é auditável e confiável

Em projetos de Machine Learning, é comum considerar que o código está adequado apenas porque ele executa sem erros e produz previsões. No entanto, fazer o código rodar é apenas o primeiro passo. Um código que funciona hoje pode tornar-se inadequado amanhã, quando novos dados forem incorporados, quando o modelo precisar ser atualizado ou quando outra pessoa precisar compreender e modificar o sistema.

Código desenvolvido apenas com foco no resultado imediato costuma misturar carregamento de dados, tratamento, modelagem e avaliação em um único fluxo. Variáveis são reutilizadas sem clareza, funções acumulam múltiplas responsabilidades e decisões importantes ficam implícitas. Esse tipo de implementação pode resolver o problema a curto prazo, mas torna-se difícil de manter, evoluir e validar.

Por outro lado, código que escala é projetado pensando no futuro. Ele permite incorporar novos dados, testar diferentes modelos e reutilizar componentes do pipeline sem a necessidade de reescrever toda a lógica. Em Machine Learning, escalar não significa apenas lidar com grandes volumes de dados, mas garantir que o experimento possa evoluir de forma controlada e segura.

Outro aspecto essencial é a auditabilidade. Em ambientes profissionais, acadêmicos ou regulatórios, é fundamental compreender como o modelo foi construído. Isso inclui saber quais dados foram utilizados, quais transformações foram aplicadas e qual versão do modelo gerou determinada previsão. Código mal estruturado dificulta ou impede essa rastreabilidade, enquanto código limpo permite reproduzir resultados, validar decisões e justificar o comportamento do modelo.

Além disso, a confiabilidade depende diretamente da clareza do código. Implementações confusas aumentam o risco de erros silenciosos, como utilizar informações do conjunto de teste no treinamento, aplicar transformações inconsistentes ou calcular métricas de forma incorreta. Esses erros não impedem a execução do sistema, mas comprometem a validade dos resultados.

Dessa forma, a diferença entre código que funciona e código bem estruturado está em sua sustentabilidade. Código limpo transforma experimentos em soluções confiáveis, facilita a colaboração entre equipes e permite que os modelos sejam compreendidos, auditados e aprimorados ao longo do tempo.

Legibilidade do código: escrever para humanos

Embora o computador execute o código, são as pessoas que precisam compreendê-lo, revisá-lo e mantê-lo. Por isso, a legibilidade é um dos princípios mais importantes em projetos de Machine Learning. Código legível comunica claramente sua intenção, permitindo que o leitor compreenda o fluxo do pipeline e as decisões implementadas.

A legibilidade depende de nomes claros, organização adequada e separação de responsabilidades. Cada parte do código deve expressar seu propósito de forma explícita, sem exigir interpretações complexas. Quando a lógica do pipeline é compreensível, torna-se mais fácil identificar erros, validar resultados e evoluir o sistema.

Em Machine Learning, o código representa diretamente o processo de aprendizado do modelo. Cada transformação aplicada aos dados e cada etapa do pipeline refletem decisões metodológicas. Quando essas decisões não são claras, a compreensão do próprio modelo torna-se comprometida.

Escrever código legível não significa escrever mais código, mas escrever código melhor estruturado. Funções com responsabilidades bem definidas, organização lógica e nomes significativos tornam o sistema mais fácil de entender, testar e manter. Isso reduz o esforço cognitivo necessário para interpretar o código e diminui o risco de erros.

Código como meio de comunicação

O código em Machine Learning não serve apenas para executar algoritmos, mas também para documentar o processo de construção do modelo. Ele deve comunicar claramente como os dados foram tratados, como o modelo foi treinado e como os resultados foram avaliados.

Nomes significativos desempenham um papel central nesse processo. Eles permitem compreender rapidamente o papel de variáveis, funções e componentes do pipeline. Quando os nomes refletem a intenção, o código torna-se autoexplicativo, reduzindo a necessidade de comentários e facilitando a manutenção.

A organização em funções pequenas e bem definidas também contribui para a clareza. Cada função deve possuir uma única responsabilidade, permitindo que o fluxo do pipeline seja compreendido de forma modular. Isso facilita testes, modificações e reutilização.

Além disso, código bem estruturado reduz a dependência de comentários. Comentários devem explicar decisões conceituais, não compensar falta de clareza na implementação. Quando o código é claro, sua própria estrutura comunica o necessário.

A legibilidade também está diretamente relacionada à confiabilidade. Código confuso dificulta a identificação de erros conceituais e compromete a validade do modelo. Estruturas bem organizadas tornam explícitas as transformações aplicadas e garantem consistência entre treinamento e avaliação.

Nomes significativos: variáveis, funções e módulos como expressão de intenção

Em Clean Code, a escolha de nomes para variáveis, funções e módulos é uma das decisões mais importantes na construção de um sistema. O nome é o primeiro elemento que comunica ao leitor o papel de cada componente e sua função dentro do pipeline. Quando os nomes são claros e expressivos, o código torna-se mais fácil de entender, revisar e manter.

Nomes significativos devem expressar o que está sendo representado e qual é sua finalidade no contexto do problema. Quando nomes genéricos ou ambíguos são utilizados, o leitor precisa analisar a implementação para compreender seu significado. Isso aumenta o esforço cognitivo, dificulta a manutenção e eleva o risco de uso incorreto dos dados ou funções.

Essa prática se aplica não apenas a variáveis, mas também a funções. O nome de uma função deve comunicar sua responsabilidade principal, não os detalhes de sua implementação. Funções bem nomeadas permitem compreender rapidamente o fluxo lógico do pipeline e facilitam a reutilização do código. Quando a função possui um nome claro, sua finalidade torna-se evidente, mesmo sem a necessidade de analisar sua estrutura interna.

Da mesma forma, a nomeação de módulos e arquivos desempenha um papel fundamental na organização do projeto. Módulos devem agrupar responsabilidades relacionadas e possuir nomes que reflitam seu conteúdo. Essa organização facilita a navegação no projeto, melhora a legibilidade e reforça a separação de responsabilidades.

Outro aspecto importante é a consistência na nomenclatura. A utilização de padrões bem definidos permite que o código seja interpretado de forma previsível. Em projetos de Machine Learning, convenções amplamente utilizadas ajudam a identificar rapidamente o papel de cada elemento do pipeline, reduzindo ambiguidades e facilitando a colaboração entre desenvolvedores(as).

Nomes significativos transformam o código em um meio de comunicação claro e eficiente. Eles permitem compreender a estrutura do sistema sem a necessidade de

interpretar sua implementação, contribuindo diretamente para a legibilidade, a auditabilidade e a confiabilidade do modelo.

Simplicidade e o princípio KISS em projetos de Machine Learning

O princípio KISS, acrônimo de “Keep It Simple, Stupid”, estabelece que soluções simples são mais eficazes, compreensíveis e sustentáveis do que soluções desnecessariamente complexas. Em projetos de Machine Learning, existe uma tendência natural de aumentar a complexidade do código à medida que novas funcionalidades, ajustes e experimentos são incorporados. No entanto, essa complexidade pode dificultar a compreensão do sistema e aumentar o risco de erros.

A simplicidade não significa ausência de rigor técnico, mas sim a capacidade de resolver um problema com o menor nível de complexidade necessário. Código simples é mais fácil de entender, testar, manter e evoluir. Ele permite que o desenvolvedor ou desenvolvedora compreenda rapidamente o fluxo do pipeline e identifique possíveis falhas conceituais ou de implementação.

Um dos principais benefícios da simplicidade é a redução da complexidade cognitiva. Quanto mais difícil é compreender um trecho de código, maior é o esforço necessário para interpretá-lo corretamente e maior é a probabilidade de erros. Código simples permite que o foco esteja na análise do problema e na interpretação dos resultados, não na tentativa de decifrar a implementação.

Além disso, a simplicidade facilita a manutenção e a evolução do sistema. Quando o código é claro e bem estruturado, modificações podem ser realizadas de forma localizada, sem comprometer outras partes do pipeline. Isso é especialmente importante em Machine Learning, em que modelos são frequentemente atualizados e experimentos são continuamente refinados.

A simplicidade também contribui para a confiabilidade do modelo. Implementações excessivamente complexas aumentam o risco de erros conceituais, como transformações inconsistentes ou uso incorreto dos dados. Um código simples torna essas etapas mais explícitas, reduzindo ambiguidades e facilitando a validação do processo.

Como aplicar o princípio KISS na prática

Aplicar o princípio KISS em projetos de Machine Learning envolve adotar uma abordagem estruturada e consciente para reduzir complexidade desnecessária. O primeiro passo é dividir o pipeline em etapas claras e bem definidas, como carregamento de dados, pré-processamento, treinamento e avaliação. Essa separação permite compreender cada etapa de forma isolada e facilita testes e modificações.

Outra prática importante é evitar abstrações prematuras. Em fases iniciais do projeto, tentar generalizar excessivamente o código pode tornar o sistema mais difícil de entender sem trazer benefícios reais. É preferível implementar soluções simples e evoluí-las gradualmente conforme novas necessidades surgem.

A simplicidade também deve orientar a escolha de modelos e ferramentas. Modelos mais complexos nem sempre produzem melhores resultados, especialmente quando os dados não justificam tal complexidade. Começar com soluções simples permite estabelecer uma base confiável e facilita a comparação entre diferentes abordagens.

Além disso, a simplicidade envolve eliminar redundâncias e lógica desnecessária; código enxuto e bem estruturado facilita a leitura e reduz o risco de inconsistências. Cada componente deve possuir uma responsabilidade clara e definida, contribuindo para um pipeline organizado e compreensível.

Uma forma prática de avaliar a simplicidade é considerar se o código pode ser compreendido rapidamente por outro desenvolvedor ou desenvolvedora. Se o entendimento exigir esforço excessivo, isso indica que há espaço para simplificação. Em Machine Learning, em que decisões técnicas impactam diretamente a validade dos resultados, simplicidade é um fator essencial para garantir confiabilidade.

Assim, o princípio KISS não limita a capacidade do sistema, mas estabelece uma base sólida para sua evolução. Código simples é mais fácil de manter, auditar e aprimorar, sendo um dos pilares fundamentais do Clean Code em Machine Learning.

O princípio KISS estabelece que soluções simples são mais eficazes e sustentáveis do que soluções desnecessariamente complexas. Em Machine Learning,

existe uma tendência natural de aumentar a complexidade à medida que o projeto evolui, mas isso pode dificultar a compreensão e aumentar o risco de erros.

Simplicidade não significa ausência de sofisticação, mas clareza na implementação. Código simples facilita leitura, manutenção e validação. Ele permite compreender rapidamente o funcionamento do pipeline e identificar possíveis problemas.

Dividir o pipeline em etapas claras e bem definidas reduz a complexidade e facilita sua evolução. Cada componente deve possuir uma responsabilidade específica, permitindo modificações localizadas e reduzindo o impacto de mudanças.

Além disso, a simplicidade também se aplica à escolha de modelos e técnicas. Modelos mais complexos nem sempre produzem melhores resultados e começar com soluções simples permite construir uma base sólida e evoluir de forma controlada.

Assim, código simples reduz a carga cognitiva, melhora a confiabilidade e facilita a auditoria do sistema.

Princípio DRY: evitar duplicação

O princípio DRY estabelece que cada regra ou transformação deve existir em um único lugar no código. A duplicação aumenta o risco de inconsistências e torna o sistema mais difícil de manter.

Em Machine Learning, esse princípio é especialmente importante, pois o comportamento do modelo depende diretamente das transformações aplicadas aos dados. Quando essas transformações são duplicadas, há maior risco de divergências entre diferentes etapas do pipeline.

Centralizar a lógica em funções ou pipelines garante consistência e facilita manutenção. Isso também melhora a reprodutibilidade e a auditabilidade dos experimentos.

Ao evitar duplicações, o código torna-se mais claro, confiável e fácil de evoluir.

Code Review: controle de qualidade em Machine Learning

Code review é o processo de revisão do código por outros desenvolvedores e desenvolvedoras antes de sua integração ao projeto. Em Machine Learning essa prática é essencial, pois o código define diretamente o comportamento do modelo.

Muitos erros em Machine Learning são silenciosos e não geram falhas técnicas, mas comprometem a validade dos resultados. O code review permite identificar esses problemas antes que o modelo seja utilizado.

Além disso, essa prática melhora a legibilidade, padroniza o código e promove troca de conhecimento entre membros da equipe.

Code review deve avaliar não apenas se o código funciona, mas se ele é claro, correto e sustentável. Quando bem aplicado, fortalece a confiabilidade dos modelos e consolida as boas práticas de Clean Code.

MERCADO, CASES E TENDÊNCIAS

Knight Capital (2012): quando código que “funciona” quebra um sistema inteiro



Figura 1 – Knight Capital
Fonte: Wall Street Millennial (2021)

Um dos casos mais emblemáticos envolvendo problemas de código, engenharia e falta de boas práticas é o da Knight Capital Group, uma das maiores empresas de trading de alta frequência dos Estados Unidos. Em agosto de 2012, a empresa colocou em produção uma atualização de software responsável por executar ordens automatizadas no mercado financeiro.

O sistema entrou em operação normalmente, sem apresentar falhas técnicas aparentes. No entanto, em menos de 45 minutos, o algoritmo começou a realizar milhares de operações incorretas, comprando e vendendo ativos de forma descontrolada. O resultado foi um prejuízo estimado em US\$ 440 milhões, levando a empresa praticamente à falência.

Investigações posteriores mostraram que o problema não foi causado por um erro matemático complexo, mas por código legado mal estruturado, falta de isolamento de funcionalidades antigas, ausência de testes adequados e falhas graves no processo de code review. Um módulo antigo, que deveria estar desativado, foi executado inadvertidamente após a atualização, pois o código não era claro, modularizado ou auditável.

Embora não fosse um sistema de Machine Learning nos moldes atuais, o caso é amplamente utilizado como referência em engenharia de software e sistemas orientados por decisão automática. Ele ilustra perfeitamente a diferença entre código que roda e código confiável, além de evidenciar como a ausência de princípios como Clean Code, simplicidade (KISS), DRY e revisão de código pode gerar impactos reais, financeiros e irreversíveis.

Esse caso mostra que, em sistemas que tomam decisões automaticamente — como pipelines modernos de Machine Learning —, o código é tão crítico quanto o modelo. Um algoritmo pode estar estatisticamente correto, mas se o código que o implementa não for claro, revisado e bem estruturado, o risco operacional se torna inaceitável.

Tendências globais na adoção de Clean Code

Nos últimos anos, o crescimento de sistemas baseados em dados e Machine Learning tem levado organizações a repensarem a forma como o código é escrito, mantido e avaliado. Se no passado o foco estava predominantemente em “fazer o sistema funcionar”, hoje há uma preocupação crescente com qualidade, sustentabilidade e governança do código. Clean Code deixou de ser uma recomendação individual e passou a ser tratado como um requisito organizacional em projetos de médio e grande porte.

Uma das principais forças por trás dessa tendência é a complexidade crescente dos sistemas. Pipelines modernos de Machine Learning envolvem múltiplas fontes de dados, transformações, modelos, retreinamentos e monitoramento contínuo. Nesse contexto, código confuso ou duplicado dificulta a evolução do sistema, aumenta custos operacionais e eleva o risco de erros silenciosos. Como resposta, empresas passaram a adotar padrões de código, revisão sistemática (code review) e arquiteturas mais simples e modulares.

Outro fator determinante é a necessidade de auditabilidade e transparência. Regulamentações, preocupações éticas e exigências de explicabilidade em sistemas automatizados tornaram essencial compreender como decisões são tomadas. Clean Code facilita esse processo ao tornar explícitas as etapas do pipeline, as

transformações aplicadas e as escolhas de modelagem. Assim, o código passa a ser parte do mecanismo de governança de sistemas de Machine Learning.

Também se observa uma forte influência da cultura DevOps e MLOps, que promove integração contínua, automação e colaboração entre equipes. Nessas abordagens, código legível, simples e padronizado é fundamental para permitir testes automatizados, versionamento de modelos e implantação segura em produção. Clean Code, nesse cenário, não é apenas uma boa prática, mas um pré-requisito técnico para escalar soluções de Machine Learning.

Por fim, há uma mudança no perfil profissional exigido pelo mercado. Cientistas de dados e engenheiros(as) de Machine Learning são cada vez mais cobrados(as) não apenas por resultados preditivos, mas pela capacidade de construir soluções robustas, mantidas ao longo do tempo e compreensíveis por outras equipes. Isso reforça a adoção de princípios como KISS, DRY, nomes significativos e code review como parte da formação técnica e da prática profissional.

O QUE VOCÊ VIU NESTA AULA?

Ao longo desta aula, exploramos como o código em projetos de Machine Learning vai muito além de simplesmente executar algoritmos ou gerar previsões. Em sistemas orientados por dados, o código é o elemento que conecta dados, hipóteses, modelos e decisões. Por isso, a qualidade do código impacta diretamente a confiabilidade, a escalabilidade e a sustentabilidade das soluções desenvolvidas.

O primeiro ponto discutido foi a diferença entre código que funciona e código bem construído. Em Machine Learning, é possível obter métricas aparentemente boas mesmo com implementações conceitualmente incorretas.

Vimos que Clean Code não é uma preocupação estética, mas uma prática essencial para garantir que o código possa ser entendido, revisado e evoluído por outras pessoas — inclusive pelo próprio autor no futuro. Em projetos colaborativos e de longo prazo, essa clareza é indispensável.

Outro pilar fundamental abordado foi a importância de nomes significativos para variáveis, funções e módulos. Bons nomes devem expressar o que algo representa, não apenas como foi implementado. A diferença entre nomes genéricos e nomes expressivos é a diferença entre um código que precisa ser decifrado e um código que se explica sozinho.

O princípio KISS (Keep It Simple, Stupid) foi apresentado como um guia para evitar complexidade desnecessária. Em Machine Learning, há uma tendência natural a criar pipelines complexos, com múltiplas abstrações, antes mesmo de compreender plenamente o problema ou os dados. Vimos que soluções simples, bem estruturadas e bem compreendidas são mais fáceis de manter, validar e explicar.

O princípio DRY (Don't Repeat Yourself) foi discutido como um dos mais importantes em pipelines de Machine Learning. Repetir lógica de pré-processamento, transformação ou avaliação em diferentes partes do código aumenta o risco de inconsistências e erros silenciosos. Cada regra deve ter uma única fonte de verdade.

Outro ponto-chave da aula foi a distinção entre código experimental e código preparado para escala. Código que escala não é apenas aquele que roda em grandes volumes de dados, mas aquele que pode ser entendido, revisado, monitorado e modificado com segurança.

Encerramos a aula discutindo a prática de code review. Em Machine Learning, o code review é essencial para identificar erros conceituais que dificilmente seriam detectados apenas pela execução do código. Revisar código é revisar decisões estatísticas, escolhas de modelagem e lógica de processamento de dados.

Esta aula mostrou que Clean Code é um pilar fundamental para quem trabalha com Machine Learning. Escrever código limpo significa tornar explícitas as decisões técnicas e estatísticas, reduzir riscos, facilitar colaboração e construir soluções sustentáveis. Ao aplicar princípios como legibilidade, nomes significativos, KISS, DRY e code review, o indivíduo desenvolvedor deixa de produzir apenas scripts experimentais e passa a construir pipelines confiáveis e profissionais.

Mais do que um conjunto de regras, Clean Code representa uma mudança de postura: escrever código pensando em pessoas, no futuro do projeto e no impacto real das decisões automatizadas. Essa base será essencial para avançar nos próximos temas ligados à Engenharia de Machine Learning e MLOps.

REFERÊNCIAS

BURKOV, A. **Machine Learning Engineering**. [S. l.]: True Positive Inc., 2020.

FOWLER, M. **Refactoring**: improving the design of existing code. 2. ed. Boston: Addison-Wesley, 2018.

GOOGLE CLOUD. **MLOps**: continuous delivery and automation pipelines in machine learning. 2026. Disponível em: <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>. Acesso em: 17 mar. 2026.

GOOGLE. **Engineering practices documentation**: code review. 2026. Disponível em: <https://google.github.io/eng-practices/review/>. Acesso em: 17 mar. 2026.

GOOGLE. **Machine Learning Engineering Best Practices**. 2026. Disponível em: <https://developers.google.com/machine-learning/guides/rules-of-ml>. Acesso em: 17 mar. 2026.

MARTIN, R. C. **Clean Architecture**: a craftsman's guide to software structure and design. Boston: Prentice Hall, 2017.

MARTIN, R. C. **Clean Code**: a handbook of agile software craftsmanship. Boston: Prentice Hall, 2008.

MCCONNELL, S. **Code Complete**. 2. ed. Redmond: Microsoft Press, 2004.

MICROSOFT. **MLOps principles**. 2026. Disponível em: <https://learn.microsoft.com/en-us/azure/machine-learning/concept-mlops>. Acesso em: 17 mar. 2026.

SCIKIT-LEARN. **Pipelines and composite estimators**. 2026. Disponível em: <https://scikit-learn.org/stable/modules/compose.html>. Acesso em: 17 mar. 2026.

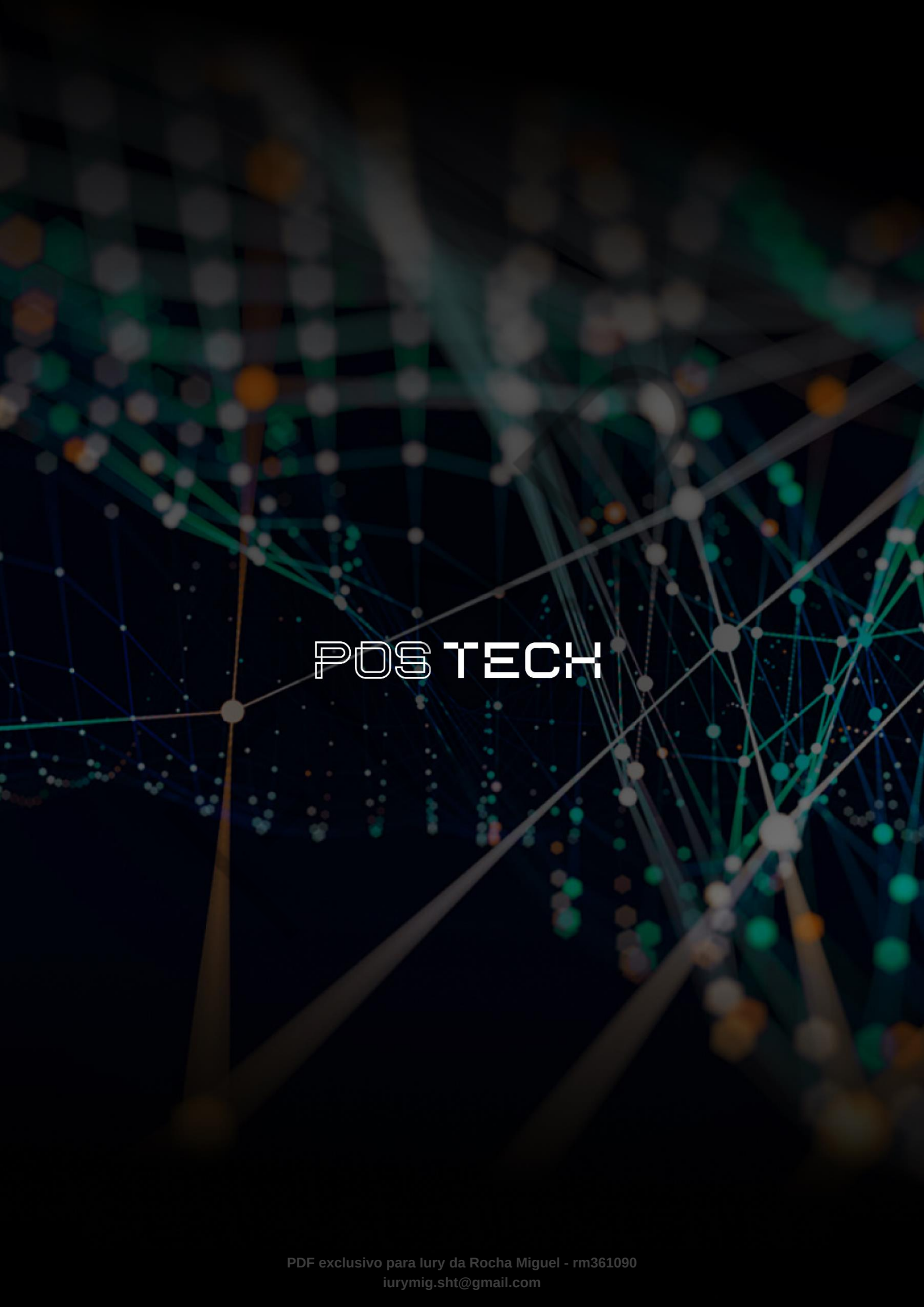
SCULLEY, D. et al. Hidden technical debt in machine learning systems. In: **Conference On Neural Information Processing Systems (NeurIPS)**, 2015. Disponível em: <https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems>. Acesso em: 17 mar. 2026.

WALL STREET MILLENNIAL. **A Hedge Fund Accidentally Used The Wrong Code, Lost \$400 Million**. 2021. Disponível em: <https://www.youtube.com/watch?v=2l1OpFJy4FM>. Acesso em: 17 mar. 2026.

PALAVRAS-CHAVE

Legibilidade. Simplicidade. Escalabilidade.

ENCERRADO



POSTECH